

MP3: Page Table Management

Mauricio Aguilar Alonso
UIN: 434002903
CSCE410: Operating Systems

Assigned Tasks

Main: Completed.

System Design

The goal of this machine problem was to implement a basic paging system that supports demand paging using the x86 two-level page table structure. The paging system makes use of the contiguous frame pools implemented in the previous machine problem to obtain physical memory. The kernel memory pool provides frames for paging structures such as the page directory and page tables, while the process memory pool provides frames for pages allocated during page faults.

When a page table is first created, the first 4 MB of memory is identity mapped so that virtual addresses correspond directly to the same physical addresses. The remainder of the virtual address space is initially left unmapped.

When the processor accesses an address that is not currently mapped, a page fault occurs. Faulting virtual addresses are divided into a page-directory index, page-table index, and page offset. If a page table for the requested address does not yet exist, a physical frame is obtained from the kernel memory pool and initialized as a new page table. A physical frame is then obtained from the process memory pool and placed into the appropriate page-table entry. After establishing a mapping, the fault handler returns and the processor can retry accessing the memory.

Code Description

The only file modified for this machine problem was `page_table.C`. The implementation uses the existing contiguous frame pool implementation to obtain physical frames when constructing paging structures and servicing page faults.

The project can be compiled using:

```
make
```

The kernel can then be run in QEMU using:

```
make run
```

page_table.C: init_paging Paging initialization stores references to the existing kernel and process frame pools along with the size of the shared memory region. References are stored as class members and are used to request physical frames when page tables or process pages need to be allocated.

page_table.C: PageTable Constructor Allocates one frame from the kernel memory pool for the page directory and another frame for the first page table. Page directory entries are initially marked as not present. The first page table is then used to identity map the first 4 MB of memory. Each entry maps a virtual page to the physical frame at the same address and is marked present and writable. The first page directory entry is configured to point to the page table. The remaining page-directory entries stay not present so that addresses outside the initial 4 MB can be allocated later through demand paging.

```

void PageTable::init_paging(ContFramePool * _kernel_mem_pool,
                             ContFramePool * _process_mem_pool,
                             const unsigned long _shared_size)
{
    kernel_mem_pool = _kernel_mem_pool;
    process_mem_pool = _process_mem_pool;
    shared_size = _shared_size;

    Console::puts("Initialized Paging System");
}

```

Figure 1: init_paging implementation

```

PageTable::PageTable()
{
    // Allocate frame for page directory
    unsigned long pd_frame = kernel_mem_pool->get_frames(1);
    page_directory = (unsigned long *) (pd_frame * PAGE_SIZE);

    // Allocate frame for first page table
    unsigned long pt_frame = kernel_mem_pool->get_frames(1);
    unsigned long *first_page_table = (unsigned long *) (pt_frame * PAGE_SIZE);

    // Mark all page directory entries as not present
    for(unsigned int i = 0; i < ENTRIES_PER_PAGE; i++)
    {
        page_directory[i] = 0x2;
    }

    // Map 4MB
    for(unsigned int i = 0; i < ENTRIES_PER_PAGE; i++)
    {
        first_page_table[i] = (i * PAGE_SIZE) | 0x3;
    }

    // Directory entry 0 points to our first page table
    page_directory[0] = (pt_frame * PAGE_SIZE) | 0x3;

    Console::puts("Constructed Page Table object\n");
}

```

Figure 2: PageTable constructor implementation

page_table.C: load The page table object is stored as the currently active page table. The address of its page directory is then written to the CR3 register. Tells the processor which page directory to use for virtual-to-physical address translation.

page_table.C: enable_paging Paging is enabled by reading the CR0 control register and setting bit 31, the paging bit. Writes back modified value to CR0. Paging state is updated.

page_table.C: handle_fault The page fault handler reads the CR2 register to obtain the virtual address that caused the page fault. The address is then separated into its page-directory index and page-table index. Corresponding page directory entry is checked to determine whether the required page table already exists. If not present, a physical frame is obtained and used to create a new page table. Entries of the new page table are initialized as writable but not present. Address of the new

```

void PageTable::load()
{
    current_page_table = this;
    write_cr3((unsigned long) page_directory);
    Console::puts("Loaded page table\n");
}

```

Figure 3: load implementation

```

void PageTable::enable_paging()
{
    unsigned long cr0 = read_cr0();

    cr0 |= 0x80000000;

    write_cr0(cr0);
    paging_enabled = 1;
    Console::puts("Enabled paging");
}

```

Figure 4: enable_paging implementation

table is placed into the corresponding page-directory entry. Once a page table exists, obtains a physical frame from the process memory pool. Address of the frame is placed into the entry for the faulting virtual address. Entry is marked present and writable. Handler returns, allows the process to retry access.

Testing

Testing was primarily performed using the testing code provided with the machine problem. Errors that would prevent compilation were largely resolved using the terminal's error feedback, although AI tools were used at some points.

Simple tests of the kernel were executed using the makefile's `make run`. The provided test involved generating a large number of page faults while accessing pages that were not initially mapped. After successful implementation. The program completed with the output:

```

DONE WRITING TO MEMORY. Now testing...
TEST PASSED
YOU CAN SAFELY TURN OFF THE MACHINE NOW.

```

```

void PageTable::handle_fault(REGS * _r)
{
    // Find the faulting address
    unsigned long fault_address = read_cr2();

    unsigned long directory_index = fault_address >> 22;
    unsigned long table_index = (fault_address >> 12) & 0x3FF;

    PageTable *pt = current_page_table;

    // Does the required page table exist?
    if ((pt->page_directory[directory_index] & 0x1) == 0)
    {
        // Allocate a frame for a new page table
        unsigned long new_pt_frame = kernel_mem_pool->get_frames(1);
        unsigned long *new_page_table =
            (unsigned long *) (new_pt_frame * PAGE_SIZE);

        // Initialize entries as writable, but not present
        for(unsigned int i = 0; i < ENTRIES_PER_PAGE; i++)
        {
            new_page_table[i] = 0x2;
        }

        // Install the new page table into the directory
        pt->page_directory[directory_index] =
            (new_pt_frame * PAGE_SIZE) | 0x3;
    }

    // Locate the page table
    unsigned long *page_table = (unsigned long *)
        (pt->page_directory[directory_index] & 0xFFFFF000);

    // Allocate the process page
    unsigned long page_frame = process_mem_pool->get_frames(1);

    // Map the faulting virtual page to it
    page_table[table_index] = (page_frame * PAGE_SIZE) | 0x3;
}

```

Figure 5: handle_fault implementation